

```

long hash;
PyObject *key;
PyObject *value;
}
Stored as:
sparse_array = [None, 1, None, None, None, 0, None, 2]
compact_array = [[-9092791511155847987, 'timmy', 'red'],
                 [-8522787127447073495, 'barry', 'green'],
                 [-6480567542315338377, 'guido', 'blue']]

```

In the latest code structure change, the compact\_array stores all the items in the order of insertion maintained in the dictionary, while sparse\_array is a half to two-thirds full array of integers (<https://morepypy.blogspot.com/2015/01/faster-more-memory-efficient-and-more.html>).

The integers themselves are of the smallest size necessary for indexing the compact\_array. So, if compact\_array has less than 256 items, then sparse\_array will be made of bytes; if less than  $2^{16}$ , it'll be two-byte integers; and so on. This design saves quite a bit of memory.

For example, on 64-bit systems we can use indexing of more than 4 billion elements but hardly ever do so; and for small dicts, the extra sparse\_array takes very little space. For example, a 100 element dict, on average for the original design on 64-bit, would be:  $100 * 12/7 * \text{WORD} * 3 \approx 4100$  bytes, while on the new design it's  $100 * 12/7 + 3 * \text{WORD} * 100 \approx 2600$  bytes. This is quite a significant saving.

With the reduced memory footprint, we can also expect better cache utilisation. Only the data layout needs to change — all the hash table algorithms stay the same, and there is no change in hash functions, search order or collision statistics. If the hash table is filled, it is copied to a new location or resized for extra space (just like once the list grows it will be resized).

## Hash collision

When the hash function gives the same value for different keys, it is known as hash collision. Here are a few methods to address hash collision.

**Open addressing:** Spread the collided values in a predictable way that lets you retrieve them later. You can use the following algorithms:

- Cuckoo hashing
- Double hashing
- Hopscotch hashing
- Linear probing
- Quadratic probing
- Robin Hood hashing

**Closed addressing:** Keep the collided values in a separate data structure to search through. This is also known as separate chaining.

**Coalesced hashing:** This combines the ideas behind both open and closed addressing into one algorithm.

### Insertion

Each item saved in a hash table contains two fields for hash mapping — a reference to the key and a reference to the value of the item. For sets, it is only the latter.

To insert the key into the dict, the hash value is calculated using the built-in `hash()` function. The hash value then performs an 'and' operation for creating a mask value so that it turns into an effective index to fit in an array (or bucket).

If we have allocated 8 blocks of memory and our hash value is 28975, we consider the bucket at index  $28975 \& 0b111 = 7$ ; however, if the dictionary has grown to require 512 blocks of memory, then the mask becomes  $0b11111111$  and we consider the bucket at index  $28975 \& 0b11111111$ .

Check if the returned index is already in use; if not, insert the key and the value into the block of memory. If already in use and the key matches the given key, update the value.

If there is a collision, i.e., the hash function has generated the same hash value for two different keys, a new index simple linear function is used, which is called probing.

hash collision (perturb)

### Search

To find the position where they should be based on the hash value, compare the hash value and key of the element in this position to the hash table to see if they are equal to the element that needs to be found. If equal, return directly; if they are not, then continue to search until a slot is found or an exception is thrown.

### Deletion

For the delete operation, Python temporarily assigns a special value to the element at this position marking it as deleted. Order is preserved and then deleted from the compact array when the hash table is resized and many entries need to be cleared. It also reindexes the sparse array, as the positions have changed.

The occurrence of hash collisions tends to reduce the speed of dictionary and set operations. In order to ensure efficiency, the dictionary and the hash table in the collection are usually guaranteed to have at least two-thirds of the remaining space.

With the continuous insertion of elements, when the remaining space is less than two-thirds, Python will regain a larger memory space and expand the hash table. However, in this case, all element positions in the table will be rearranged.